

Lesson 2 — Your First Machine Learning Model

In Lesson 1, you learned:

- `x` = features/input
- `y` = target/output
- Regression → predicts numbers
- `fit()` → trains a model
- `predict()` → makes predictions
- `random_state` → makes the random split reproducible

Now we're going to actually train our first model.

We'll use Linear Regression because it's one of the simplest ML algorithms and teaches the basic Scikit-learn workflow.

1. What is Linear Regression?

Suppose we have house data:

Area	Price
500	₹30L
1000	₹55L
1500	₹80L
2000	₹105L

We can imagine that there is a relationship:

Area increases
↓
Price generally increases

Linear Regression tries to find the best-fitting line through the data.

Mathematically:

$$y = mx + b$$

where:

- `x` = input
- `y` = predicted output
- `m` = slope
- `b` = intercept

With multiple features, the idea becomes:

$$y = b + w1*x1 + w2*x2 + \dots + wn*xn$$

Don't worry about the mathematics yet. The important idea is:

The model learns the relationship between `X` and `y`.

2. Load Our Dataset

In your Jupyter Notebook:

```
from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(return_X_y=True)
```

Remember:

```
X → house information
y → house value
```

Check:

```
print(X.shape)
print(y.shape)
```

You should get approximately:

```
(20640, 8)
(20640,)
```

So:

```
20,640 examples
8 features
```

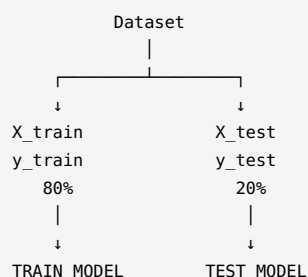
3. Split the Data

We don't want to train and test on the same data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

This gives:



Check:

```
print(X_train.shape)
print(X_test.shape)
```

You should see roughly:

```
(16512, 8)
(4128, 8)
```

4. Import Linear Regression

Now:

```
from sklearn.linear_model import LinearRegression
```

We're importing the `LinearRegression` algorithm from Scikit-learn.

5. Create the Model

```
model = LinearRegression()
```

At this point, the model hasn't learned anything yet.

Think of it like creating an empty brain:

```
model = LinearRegression()

↓

Empty model
No learning yet
```

6. Train the Model

Now the important part:

```
model.fit(X_train, y_train)
```

This tells the model:

“Here is the training data. Learn the relationship between `X_train` and `y_train`.”

Before:

```
model
↓
doesn't know anything
```

After:

```
model.fit(X_train, y_train)
↓
model learns
↓
trained model
```

7. Make Predictions

Now give the model data it hasn't seen during training:

```
predictions = model.predict(X_test)
```

The model produces predicted prices:

```
X_test  
↓  
trained model  
↓  
predictions
```

Look at them:

```
print(predictions[:5])
```

You'll get something similar to:

```
[4.23 2.91 3.15 1.72 2.84]
```

These are the model's predictions.

8. Compare Prediction With Actual Value

Remember:

```
y_test
```

contains the real values.

So:

```
print(predictions[:5])  
print(y_test[:5])
```

You might see:

```
Predicted:  
[4.23 2.91 3.15 1.72 2.84]  
  
Actual:  
[4.52 3.01 3.24 1.65 2.91]
```

They're not exactly the same.

That's normal.

The model isn't perfect.

9. How Do We Know If the Model Is Good?

This is where evaluation metrics come in.

For regression, one common metric is:

R² Score

Import:

```
from sklearn.metrics import r2_score
```

Then:

```
score = r2_score(y_test, predictions)

print(score)
```

You might get something like:

```
0.60
```

The exact result can vary depending on the data and model.

10. What Does R^2 Mean?

Very roughly:

```
 $R^2 = 1.0$ 
```

means the model explains the data extremely well.

```
 $R^2 = 0$ 
```

means the model isn't doing better than a simple baseline based on the target mean.

It can also be negative, meaning the model performs worse than that baseline.

For now, remember:

Higher R^2 is generally better for regression.

We'll learn R^2 properly in the next lesson.

11. Complete Code

Your entire Lesson 2 notebook can contain:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

Then:

```
X, y = fetch_california_housing(return_X_y=True)
```

Split:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42)
```

```
)
```

Create model:

```
model = LinearRegression()
```

Train:

```
model.fit(X_train, y_train)
```

Predict:

```
predictions = model.predict(X_test)
```

Evaluate:

```
score = r2_score(y_test, predictions)

print("R2 Score:", score)
```

Understand These 5 Lines

These five lines are extremely important:

```
model = LinearRegression()
```

Create the model.

```
model.fit(X_train, y_train)
```

Train the model.

```
predictions = model.predict(X_test)
```

Make predictions.

```
score = r2_score(y_test, predictions)
```

Measure how well it performed.

```
print(score)
```

See the result.

☆ One More Important Concept

You might wonder:

Why don't we do this?

```
model.fit(X_test, y_test)
```

Because `X_test` is supposed to represent unseen data.

The correct flow is:



That's the foundation of almost everything you'll do in Scikit-learn.

Lesson 2 Exercise

Don't just copy the code. Try these yourself.

Exercise 1

Print:

```
X_train.shape
X_test.shape
y_train.shape
y_test.shape
```

Try to understand why each has that shape.

Exercise 2

Print the first 10 predictions:

```
predictions[:10]
```

Exercise 3

Print the first 10 actual values:

```
y_test[:10]
```

Exercise 4

Compare them:

```
for predicted, actual in zip(predictions[:10], y_test[:10]):
    print("Predicted:", predicted, "Actual:", actual)
```

What you should understand before Lesson 3

Make sure these are clear:

```
X_train → data used for learning  
y_train → correct answers used during learning  
X_test → unseen input data  
y_test → correct answers for the unseen data  
fit() → learn  
predict() → predict  
metric → measure performance
```

Lesson 3 will be: Regression Evaluation — MAE, MSE, RMSE and R^2 , where we'll understand exactly what that `0.60` score means and why simply looking at predictions isn't enough.